

Database Backup Procedure

wiki/sre/database-backup · Updated 2021-11-09

Owner: Site Reliability Engineering, Data Engineering · POC: Luis Romero, Database Reliability Engineer

Overview

This procedure describes how the Northwind primary database is backed up and how to restore from a backup. It is maintained by the SRE and Data teams. Backups are our last line of defence against data loss, so the process must be reliable and regularly verified.

A backup that has never been tested is not a backup. In addition to taking backups, we periodically practise restoring them.

Backup strategy

We take full nightly backups and retain enough history to recover from a problem that is not noticed immediately. The strategy balances recovery needs against storage cost and keeps the process simple enough to run reliably every night.

How backups run

A nightly cron job runs mysqldump against the primary database and copies the archive to the on-prem NAS at nas.corp.internal. The job runs during the low-traffic window to limit impact on production.

- The dump captures a consistent snapshot of the database.

- The archive is compressed and timestamped.
- The archive is copied to the on-prem NAS for retention.

Scheduling

The job is scheduled for the quietest part of the night to minimise impact on production. If the window shifts because of traffic patterns, update the schedule rather than letting backups run during peak load.

Verification

Verify the backup size each morning. A sudden drop in size usually means the dump failed or was truncated. If the size looks wrong, investigate before the next nightly run so you do not stack up multiple bad backups.

Retention

Backups are retained according to the data retention policy and older archives are purged when they expire. Keep enough history to recover from a problem that is not noticed immediately.

Storage and purging

Expired archives are purged automatically so storage does not grow without bound. Confirm the purge job is running; a full NAS causes the nightly backup to fail silently.

Restoring

To restore, copy the dump from the NAS and pipe it into the mysql client on the standby host. Always restore to the standby first and validate before

promoting it, rather than restoring directly over production.

- Copy the chosen archive from the NAS to the standby host.
- Decompress the archive.
- Pipe the dump into the mysql client on the standby.
- Validate row counts and spot-check critical tables.
- Promote the standby only after validation passes.

Validating a restore

A restore is not complete until it is validated. Compare row counts against expectations, spot-check a few critical tables, and confirm the application can connect and read. Only then consider promoting the standby.

Testing restores

Schedule a restore drill on a regular cadence. A drill proves the backups are usable and keeps the team practised, so that a real recovery is routine rather than improvised. Record each drill, including how long the restore took, so recovery-time expectations are realistic.

Worked example

During a drill, an engineer copies the most recent dump from the on-prem NAS to the standby host, decompresses it, and pipes it into the mysql client on the standby. They validate row counts against the previous day's figures, spot-check the orders and customers tables, and confirm a test application

can read from the standby. The drill takes forty minutes, which they record as the realistic recovery time.

Frequently asked questions

How often do backups run? Nightly, during the low-traffic window.

Where are backups stored? On the on-prem NAS at `nas.corp.internal`.

Where do I restore to? The standby host first; validate before promoting, never restore straight over production.

Common mistakes

- Restoring directly over production instead of to the standby first.
- Never testing a restore and discovering at recovery time that the backups are unusable.
- Ignoring a sudden drop in backup size.
- Letting the NAS fill up so nightly backups fail silently.

Responsibilities

The SRE team owns the backup jobs and the restore procedure. The Data team owns validation of restored data. Failures of the nightly job page the on-call engineer.

Related documents

See the Data Retention Policy for how long backups are kept, the On-Call Runbook for what happens when a backup job fails, and the Monitoring and Alerting Setup for how failures are detected.

Recovery objectives

Two numbers define what our backups must achieve. The recovery point objective is how much data we can afford to lose, measured in time. With nightly backups, our worst-case recovery point is close to twenty-four hours, so anything that needs a tighter window requires additional protection such as replication. The recovery time objective is how long a restore is allowed to take, and restore drills are what keep it honest.

- Know the recovery point and recovery time expected for the database before relying on this process.
- If a tighter recovery point is required, combine backups with replication; backups alone are not enough.
- Record restore-drill durations so the recovery time reflects reality, not hope.

Offsite and resilience

A backup that sits next to the thing it protects is vulnerable to the same failure. Treat offsite resilience as part of the backup strategy, not an afterthought.

- Ensure at least one copy is isolated from a single-site failure.

- Protect backups from accidental or malicious deletion.
- Encrypt archives so that a stolen copy is not a data breach.
- Monitor the backup job and page the on-call engineer on failure.

Restore drill log

Every restore drill should be recorded so the team can see that backups are regularly proven and how long recovery actually takes. The log is also useful evidence during audits and incident reviews.

- Record the date of each drill and who performed it.
- Note which backup was restored and to which host.
- Record how long the restore and validation took end to end.
- Capture any problems encountered and the follow-up actions.

Review the log periodically. A drill that has not happened in a long time, or that keeps hitting the same problem, is a signal to invest in the recovery process before a real incident forces the issue.